

Introducción y Objetivos.

El presente proyecto expone el diseño, desarrollo e implementación de una base de datos relacional para una plataforma de comercio electrónico. El sistema permite gestionar de manera eficiente el inventario, los clientes, el ciclo de vida de los pedidos y la retroalimentación de los usuarios mediante reseñas.

Objetivos del Proyecto:

- Diseñar un modelo relacional eficiente que explore la redundancia de datos.
- Garantizar la integridad de la información mediante restricciones de validación.
- Desarrollar lógica de negocio automatizada para asegurar el cumplimiento de las reglas.

Análisis del modelo (3NF).

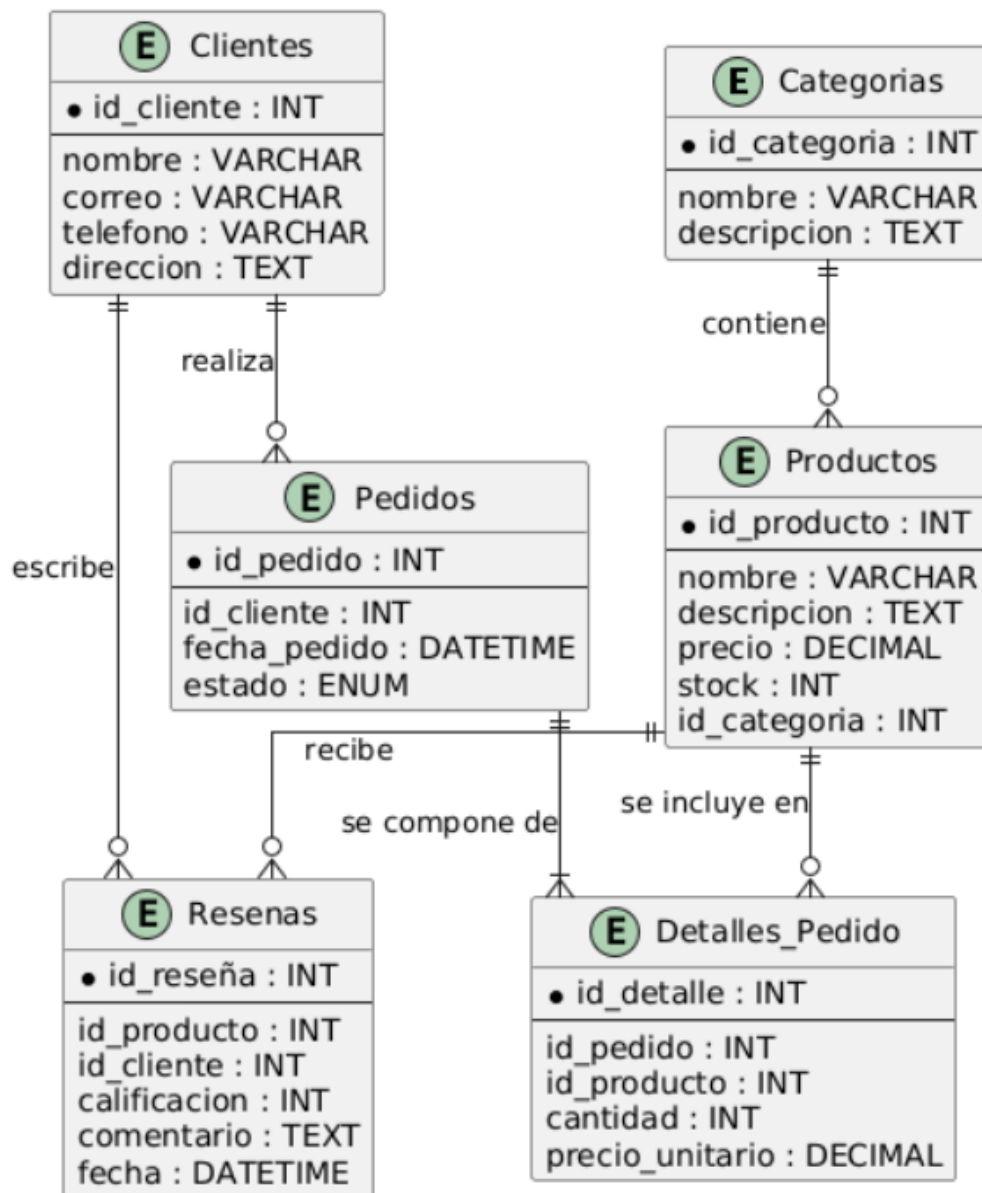
El modelo ha sido estructurado siguiendo las reglas de normalización para asegurar la integridad de los datos:

- **Primera Forma Normal (1NF):** Todos los atributos son atómicos; se eliminaron grupos repetidos.
- **Segunda Forma Normal (2NF):** Los atributos no clave dependen funcionalmente de la clave primaria.
- **Tercera Forma Normal (3NF):** Se eliminaron dependencias transitivas. Por ejemplo, la información de las categorías se encuentra en su propia tabla, evitando anomalías en el catálogo de productos.

Diccionario de datos:

Tabla	Descripción	Llave Principal (PK)	Llave Foránea (FK)
Categorías	Clasificación de productos	id_categoria	-
Cientes	Datos de usuarios	id_cliente	-
Productos	Inventario disponible	id_producto	id_categoria
Pedidos	Cabecera de transacciones	id_pedido	id_cliente
Detalles_Pedido	Ítems individuales por pedido	id_detalle	id_pedido, id_producto
Resenas	Calificaciones de clientes	id_reseña	id_producto, id_cliente

Diagrama:



CONCLUSIONES Y LECCIONES APRENDIDAS

- **Integridad:** El uso de restricciones directamente en la base de datos es la capa de seguridad más efectiva contra errores humanos.
- **Optimización:** La implementación de índices permite que las consultas complejas se ejecuten en milisegundos incluso al escalar el volumen de datos.
- **Mantenibilidad:** El uso de procedimientos almacenados centraliza la lógica del negocio, haciendo que el sistema sea fácil de actualizar sin alterar el código de la aplicación.
- **Modularidad y Reutilización:** El uso de procedimientos almacenados permite encapsular la lógica de negocio repetitiva. Esto significa que operaciones complejas, como validar el stock y restar inventario, se hacen con una sola línea de código desde cualquier aplicación externa.
- **Seguridad y Control de Accesos:** Al gestionar las reglas de negocio en el servidor (como impedir reseñas si no hay compra), la base de datos se defiende sola de accesos malintencionados o de errores que puedan venir desde la página web o app móvil.
- **Consistencia Transaccional:** La implementación de bloques `START TRANSACTION` y `COMMIT` garantiza que los pedidos nunca queden a medias. Si el sistema se apaga o falla a mitad de una compra, todo se cancela de forma automática evitando pérdidas o descuadres de inventario.

Guía de ejecución:

El sistema fue implementado exitosamente en un entorno MySQL Workbench utilizando un diseño modular dividido en tres fases principales:

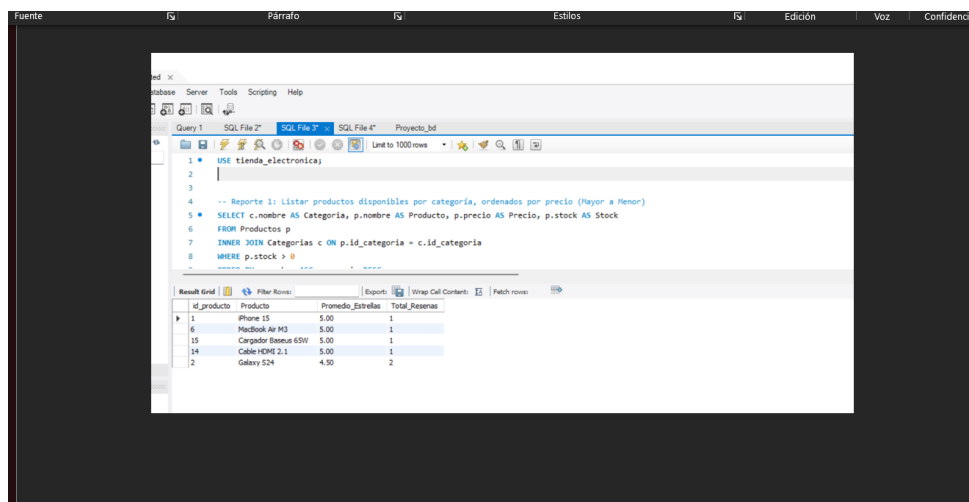
1. **DDL:** Creación de las 6 tablas principales respetando las restricciones de integridad relacional (PRIMARY KEY, FOREIGN KEY, CHECK y UNIQUE) junto con la declaración de 3 índices de rendimiento para optimizar las búsquedas.
2. **DML:** Inserción masiva de registros de prueba (30 productos, 15 clientes, 20 pedidos, 25 detalles y 10 reseñas) para garantizar un entorno de pruebas realista y comprobar la consistencia de las llaves foráneas.
3. **Lógica del negocio:** Automatización del sistema a través de 8 procedimientos almacenados que controlan de forma segura la consistencia de los inventarios y validan restricciones complejas, como impedir que un usuario publique opiniones sobre productos que no ha comprado por ejemplo.

Prueba 1: Control de Reseñas Fraudulentas

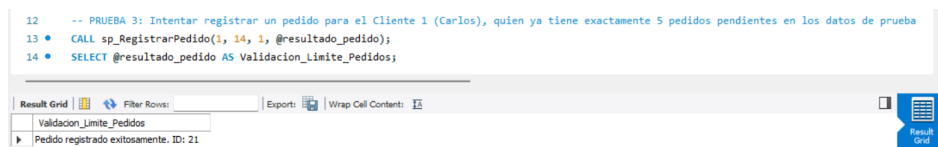
usuario publique opiniones sobre productos que no ha comprado por ejemplo.



Prueba 2: Reporte de Inventario Crítico



Prueba 3: Restricción de Pedidos Pendientes por Cliente



Prueba 4: Flujo Completo de Venta y Descuento de Stock

```
17
18 -- PRUEBA 4: El Cliente 2 (Ana) compra un iPhone 15 (Producto 1).
19 -- Ella no tiene pedidos pendientes acumulados y hay stock disponible.
20 • CALL sp_RegistrarPedido(2, 1, 1, @resultado_exito);
21 • SELECT @resultado_exito AS Procesamiento_Pedido_Exitoso;
22
23 -- Verificamos que el stock del iPhone 15 bajó de 15 a 14 unidades
24 • SELECT id_producto, nombre, stock FROM Productos WHERE id_producto = 1;
```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:

	id_producto	nombre	stock
▶	1	iPhone 15	14
•	NULL	NULL	NULL